SPM Module 2: Partitioned Hash Join with Duplicates

Gabriele Marsili

1 Parallelization Strategy

The partitioned hash join pipeline consists of five phases: *Histogram*, *Prefix Sum*, *Scatter*, *Join Local*, and a final sequential *Accumulation*. Each phase has distinct computational and memory characteristics that determine whether parallelism is beneficial and which strategy to adopt.

Thread Team and Barrier Synchronization

A single team of k threads is launched once and reused across all phases via `std::barrier` (C++20). An alternative design would use a thread pool with a task queue, but it would introduce unnecessary indirection: because all threads must finish each phase before the next one can begin — due to data dependencies between histogram, prefix sum, scatter, and join — a full barrier is required at every phase boundary regardless. The barrier approach directly models this bulk-synchronous structure and avoids the overhead of thread pool bookkeeping (task submission, queue polling, wakeup latency). Spawning and joining threads at each phase boundary would inflate the sequential component f in Amdahl's model by $O(k)$ spawn/join cycles per phase. The barrier completion function runs a single delegate thread between phases to perform the sequential prefix sum and to compute per-thread scatter offsets from the local histograms.

Adaptive thread count. To prevent synchronisation overhead from dominating small workloads, the actual thread count k is:

$$k = \min(k_{\max},\ \lfloor \min(NR, NS) \,/\, T_{\min} \rfloor)$$

where $k_{\max}$ is the user-requested count and $T_{\min} = 8\,192$ records is the minimum per-thread work threshold. Below this threshold, the cost of barrier synchronisation and cache-line traffic between threads exceeds the parallelism gain. For $NR = 10^7$ and $k_{\max} = 20$ the formula gives $k = \min(20, 1220) = 20$; for $NR = 10^4$ it gives $k = \min(20, 1) = 1$, falling back to a single-threaded execution with no synchronisation overhead.

Histogram (Phases 0 and 2)

Each thread processes a contiguous block of $\lceil N/k \rceil$ records from relation R (or S), incrementing its own private histogram of size P. Thread-local histograms eliminate all write contention. After the barrier, the completion delegate merges the k local histograms into a global one in $O(P \cdot k)$ time, then runs the prefix sum $O(P)$ and computes per-thread scatter offsets.

Static block distribution yields good spatial locality: each thread reads a contiguous slice of the input array, maximising cache line utilisation. The dominant cost of this phase is a sequential scan of $N \times 8\,\mathrm{B}$ of input — a purely memory-bandwidth-bound operation with very low arithmetic intensity ($I \approx 0.125\,\mathrm{FLOP/byte}$). As a consequence, the histogram phase scales poorly with thread count (Section 3).

Prefix Sum

The prefix sum over the P-element global histogram is performed sequentially by the barrier completion delegate. With $P \leq 1024$ this takes at most a few microseconds and represents a negligible fraction of total time ($< 0.1\%$ in all measurements). The theoretical span of a parallel prefix sum is $O(\log P)$, but the overhead of synchronising threads for such a small workload would dwarf any gain.

Scatter (Phases 1 and 3)

Scatter rearranges records into partition-contiguous output arrays. The key property enabling lock-free parallelism is that the per-thread scatter offsets are computed during the barrier: thread t starts writing partition p at offset

$$\mathrm{off}[t][p] = \mathrm{global\_begin}[p] + \sum_{j=0}^{t-1} \mathrm{local\_hist}[j][p].$$

Each thread thus writes to a non-overlapping, predetermined region of the output array with no synchronisation needed at runtime:

```
auto cursor = offsets[t]; // local copy
for (size_t i = r_start; i < r_end; ++i) {
    uint32_t pid = hash_key(R[i].key, shift);
    out_R[cursor[pid]++] = R[i];
}
```

Scatter is memory-bound (random writes driven by hash values), but the absence of any contention allows good parallel throughput.

Join Local (Phase 4)

Each partition can be joined independently: a flat open-addressing hash table `FlatCountMap` is built on the R slice, then the S slice is probed. This is an embarrassingly parallel workload. Partitions are distributed cyclically: thread t owns partitions t, $t{+}k$, $t{+}2k, \ldots$ — zero scheduling overhead, no atomics.

```
for (uint32_t pid = t; pid < P; pid += nt) {
    FlatCountMap countR(hist_R[pid]);
    for (size_t i = rb; i < re; ++i)
        countR.increment(out_R[i].key);
    for (size_t i = sb; i < se; ++i) {
        uint32_t m = countR.count(out_S[i].key);
        if (m) { local.join_count += m; ... }
    }
}
```

For the Fibonacci partitioning hash, partition sizes are statistically near-equal, so cyclic assignment achieves the same load balance as LPT scheduling with zero per-partition synchronisation. Per-thread result accumulators are padded to 64 bytes (`alignas(64)`) to prevent false sharing. After the barrier, the main thread reduces the partial results.

Summary

Phase	Strategy	Bottleneck
Histogram	Thread-local + merge	Memory BW
Prefix Sum	Sequential	Negligible
Scatter	Lock-free offsets	Memory BW
Join Local	Cyclic distribution	Compute
Accumulate	Sequential	Negligible

2 Correctness Verification

Correctness is verified at two levels. For small inputs ($NR \leq 500$, $NS \leq 500$), both the sequential and parallel binaries automatically run a naive $O(N^2)$ reference join and compare `join_count`, `checksum1`, and `checksum2`:

```
./hashjoin_par -nr 50 -ns 80 -seed 13 \
  -max-key 8 -p 4 -t 4
# output: naive_verify=PASS
```

For large inputs, the same three output values produced by the sequential and parallel binaries are compared across all tested thread counts. Since `checksum1` and `checksum2` use independent hash multipliers, the probability of a false positive is negligible. The verification is outside the measured region and does not affect reported timings.

3 Performance Results

All experiments were run on a node of the `spmcluster`: Intel Xeon E5-2640 v2 @ 2.00 GHz, 2 sockets × 8 physical cores (16 physical, 32 with Hyper-Threading), NUMA architecture. Default parameters: `seed=42`, `max_key=1M`, `P=128`, best of 3 repetitions.

Strong Scaling

p	NR=10M		NR=20M	
	$S(p)$	$E(p)$	$S(p)$	$E(p)$
seq	1.00	100%	1.00	100%
1	1.04	104%	1.02	102%
2	1.44	72%	1.43	72%
4	2.62	66%	2.52	63%
8	4.57	57%	4.43	55%
10	5.37	54%	5.39	54%
14	6.67	48%	6.81	49%
20	7.12	36%	7.53	38%
24	7.99	33%	8.56	36%
32	8.29	26%	9.27	29%

Weak Scaling

Each thread processes 1M records of R (and 2M of S). WSE = $T(1)/T(p)$; ideal is 1.0. The $p = 1$ parallel baseline (≈ 78 ms on NR=1M) incurs negligible overhead from the barrier infrastructure.

p	NR	T (ms)	WSE
1	1M	78	1.00
2	2M	106	0.74
4	4M	112	0.70
8	8M	131	0.60
10	10M	145	0.54
14	14M	155	0.51
20	20M	206	0.38
24	24M	209	0.37
32	32M	238	0.33

Phase Breakdown

Phase	T_{seq}	$S(20)$	$S(24)$	$S(32)$
Histogram R+S	51 ms	2.4×	3.2×	2.7×
Scatter R+S	410 ms	9.7×	11.3×	13.7×
Join Local	300 ms	6.9×	8.5×	7.8×
End-to-end	782 ms	7.1×	8.0×	8.3×

End-to-end includes memory allocation for partitioned arrays (∼480 MB); individual phase times are from the pipeline's internal timer.

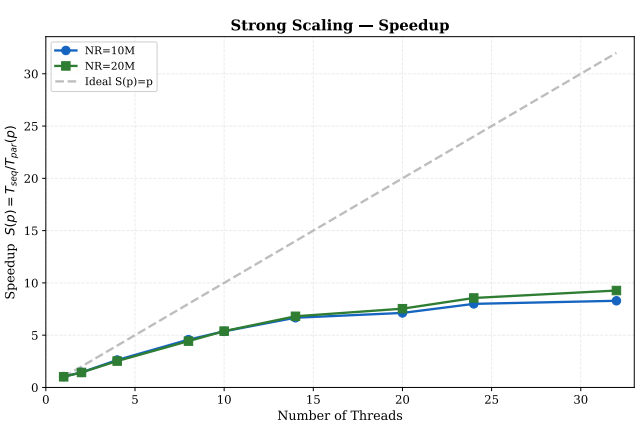
4 Discussion

Strong Scaling and Amdahl's Law

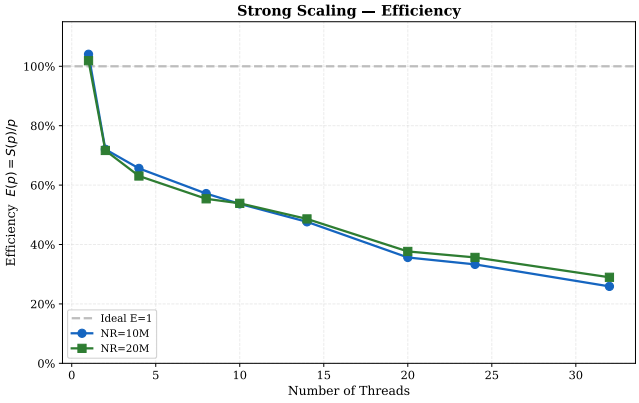
Fitting Amdahl's law $S(p) = 1/(f + (1-f)/p)$ to the full dataset (including $p = 24, 32$) yields $f \approx 0.083$, giving $S_\infty = 1/f \approx 12.1\times$. The measured peak is 8.29× (NR=10M, $p = 32$) and 9.27× (NR=20M, $p = 32$). The gap between the Amdahl prediction and the measured peak is explained by memory-bandwidth saturation: as threads increase, DRAM bandwidth is the binding constraint rather than the serial fraction.

The efficiency drops from 72% at $p = 2$ to 26% at $p = 32$ (NR=10M). Beyond $p = 14$ physical cores the end-to-end time improves by less than 15% per doubling of threads, making $p = 14$–20 the practical sweet spot on this hardware.

Hyper-Threading regime ($p = 20$–32). The phase breakdown reveals a counter-intuitive split at $p > 16$: scatter continues to improve (R+S combined: 9.7× → 13.7× from $p = 20$ to $p = 32$) while join peaks at $p = 14$ (8.1×) then regresses to 6.9× at $p = 20$ before recovering to 7.8× at $p = 32$. Scatter is memory-bandwidth-bound — HT threads increase memory-level parallelism, so additional logical cores help. Join Local (hash-table lookups with `FlatCountMap`) is compute-bound: HT pairs share execution ports on the same physical core, so competing hash-lookup streams contend for ALU and L1/L2 resources rather than adding capacity. The net result is a moderate end-to-end gain (7.1× → 8.3×) despite the scatter improvement.



(a) Speedup $S(p) = T_{\text{seq}}/T_{\text{par}}(p)$.



(b) Efficiency $E(p) = S(p)/p$, declining from 72% at $p = 2$ to 26% at $p = 32$. Shaded rows: HT threads ($p > 16$).

Figure 1: Strong scaling — both problem sizes (NR=10M and NR=20M) follow nearly identical profiles.

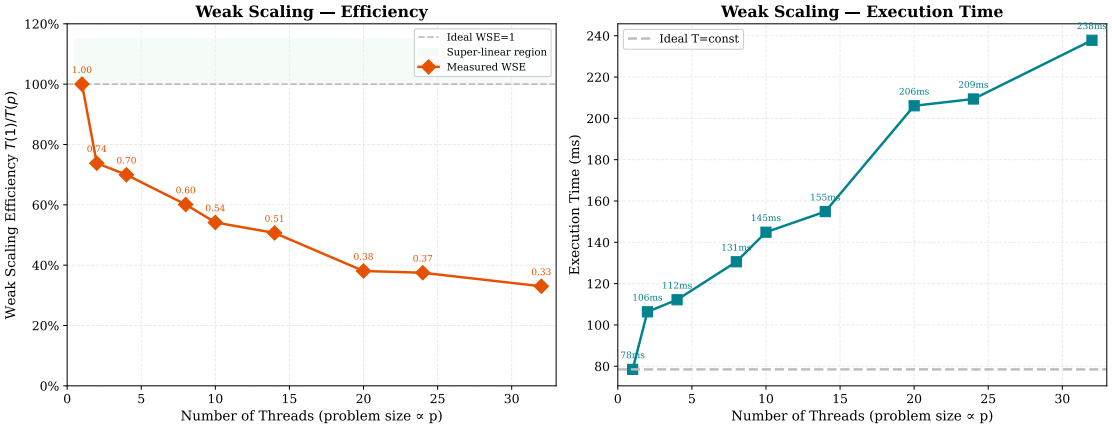


Figure 2: Weak scaling: efficiency (left) and absolute execution time (right). WSE declines monotonically; the dominant cost is scatter, whose random-write pattern saturates DRAM bandwidth as problem size grows with thread count (see §4).

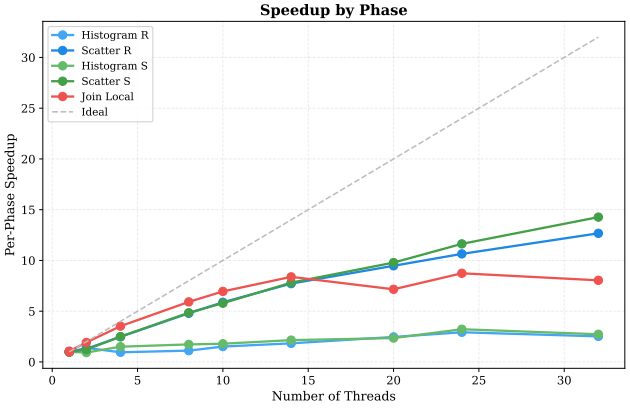


Figure 3: Per-phase speedup at 20 threads. Join Local and Scatter scale well; Histogram is memory-bound.

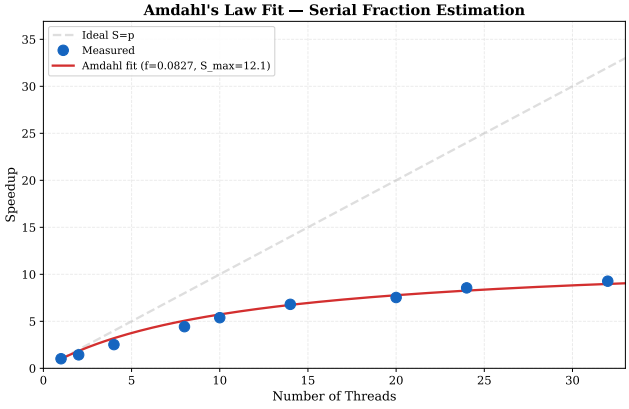


Figure 4: Amdahl's law fit (NR=20M). Estimated serial fraction $f \approx 0.083$; theoretical $S_\infty \approx 12.1\times$. The measured peak falls below the prediction due to memory-bandwidth saturation.

Weak Scaling and WSE Interpretation

The $p = 1$ parallel binary takes ≈ 78 ms on NR=1M with negligible overhead from the barrier infrastructure. WSE $= T(1)/T(p)$ decreases monotonically: 0.74 at $p = 2$, 0.54 at $p = 10$, 0.33 at $p = 32$.

The primary cause is scatter: as the problem size grows proportionally with thread count, the random-write pattern of scatter generates an increasing volume of DRAM traffic that cannot be absorbed by more cores (bandwidth is shared, not replicated). Histogram and join scale somewhat better because their access patterns benefit from per-thread private structures (thread-local histograms) or fit within the aggregate L3 capacity.

The WSE decay accelerates at $p > 16$ due to NUMA penalties: with two sockets, a larger fraction of each thread's working set resides on the remote node.

Phase-Level Analysis

Histogram (2.4–3.2× at $p = 20$–32). Arithmetic intensity $I \approx 0.125$ FLOP/byte places this phase firmly in the memory-bandwidth-limited region of the roofline model. Histograms account for only ∼7% of sequential time, so their poor scalability affects the overall bound by $< 5\%$ (Amdahl).

Scatter (9.7–13.7× at $p = 20$–32). Scatter now dominates sequential time (54%). The lock-free pre-computed-offset design eliminates all contention; speedup is limited only by DRAM write bandwidth. The continuing improvement through HT threads (Scatter improving 9.7 → 13.7× from $p = 20$ to $p = 32$) confirms this phase benefits from increased memory-level parallelism.

Join Local (peaks at 8.1× for $p = 14$, 7.8× at $p = 32$). With $P = 128$ and NR=10M, each partition holds ≈ 78K records; the `FlatCountMap` occupies ≈ 4 MB (next-power-of-two above 2×78K: 262K slots × 16 B), well above L2 (256 KB) but fitting in the shared 20 MB L3. Cyclic assignment provides near-zero scheduling overhead for the uniform Fibonacci hash. The phase scales near-linearly to $p = 14$ (8.1×); at $p > 16$ HT threads share execution ports, causing the regression described above.

Partition Count Sensitivity

With $P \in [16, 4096]$ (at $p = 20$, NR=10M), execution time reaches a minimum at $P = 512$–1024 (≈ 86 ms), with a variation of $< 1.6\times$ across the full range. At $P = 16$ each partition holds ≈ 625K records and the `FlatCountMap` occupies ≈ 32 MB, spilling into DRAM. The optimum at $P = 512$ corresponds to ≈ 20K records/partition, with the hash table (≈ 1 MB) fitting well within the shared 20 MB L3. Beyond $P = 1024$ scatter overhead grows slightly, raising time to 94 ms at $P = 4096$. A non-monotonic bump at $P = 32$ (130 ms, slower than $P = 16$) reflects a transient cache-occupancy regime between the DRAM-spill and L3-fit regions. The broad plateau around $P \in [256, 1024]$ confirms that partition count is not a critical tuning parameter.

Duplicate Density

Sequential time is essentially constant for `max_key` ≤ 1M (≈ 700–770 ms): with few distinct keys the per-partition `FlatCountMap` has few occupied slots and most probes terminate immediately. At `max_key`=10M (near-unique keys) the hash table is fully populated, increasing sequential time by +31% (≈ 700 ms → 915 ms) due to increased L3 pressure during the join phase. The parallel speedup remains stable across all duplicate densities (≈ 6.4–7.5× at $p = 20$), confirming that partition independence is unaffected by key distribution.

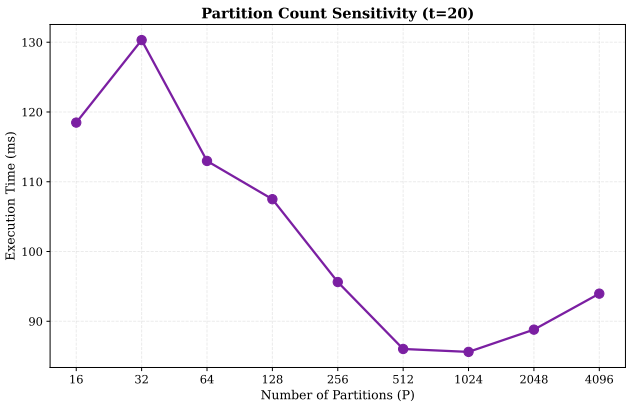


Figure 5: Partition count sensitivity ($p = 20$, NR=10M). Minimum at $P = 512$–1024; non-monotonic bump at $P = 32$ reflects a cache-occupancy transition.

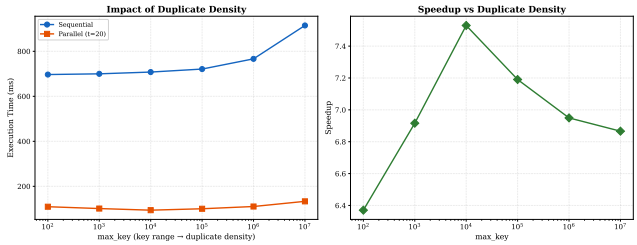


Figure 6: Impact of duplicate density ($p = 20$, NR=10M). Left: absolute execution time (seq and par). Right: speedup vs. `max_key`. Both seq and par time grow with `max_key` due to hash-table pressure; speedup remains stable (6.4–7.5×).